

数値計算 非線型計画法 1,2

3S-29 辻本純平

1. 実行結果

(ア) 黄金分割法

```
vscode → /workspaces/numerical-calculation/src (main) $ ./goldsect  
func(0.381966)  
  1:  f(0.381966) = 0.00461938  
func(0.618034)  
  2:  f(0.618034) = 0.0924367  
func(0.236068)  
  3:  f(0.236068) = 0.0060734  
func(0.472136)  
  4:  f(0.472136) = 0.025007  
func(0.326238)  
  5:  f(0.326238) = 0.000149767  
func(0.291796)  
  6:  f(0.291796) = 0.000493015  
func(0.347524)  
  7:  f(0.347524) = 0.00112387  
func(0.313082)  
  8:  f(0.313082) = 8.42166e-07  
func(0.304952)  
  9:  f(0.304952) = 8.1872e-05  
func(0.318107)  
 10:  f(0.318107) = 1.68699e-05  
func(0.309977)  
 11:  f(0.309977) = 1.61871e-05  
func(0.315002)  
 12:  f(0.315002) = 1.00337e-06  
func(0.311896)  
 13:  f(0.311896) = 4.42655e-06  
func(0.313815)  
 14:  f(0.313815) = 3.4062e-08
```

```
func(0.314269)
15: f(0.314269) = 7.21164e-08
func(0.313535)
16: f(0.313535) = 2.15846e-07
func(0.313989)
17: f(0.313989) = 1.31987e-10
func(0.314095)
18: f(0.314095) = 9.11543e-09
func(0.313922)
19: f(0.313922) = 6.02106e-09
func(0.314029)
20: f(0.314029) = 8.62468e-10
func(0.313963)
21: f(0.313963) = 1.34977e-09
func(0.314004)
22: f(0.314004) = 1.69513e-11
func(0.314014)
23: f(0.314014) = 1.89395e-10
func(0.313998)
24: f(0.313998) = 3.3991e-12
func(0.313994)
25: f(0.313994) = 3.05552e-11
func(0.314)
26: f(0.314) = 1.87647e-13
func(0.314002)
27: f(0.314002) = 3.38689e-12
x = 0.314
vscode → /workspaces/numerical-calculation/src (main) $
```

最小値付近で 26 回の関数評価 を行い，収束値として $x \doteq 0.314$ を得た．

(イ) フィボナッチ探索法

```
vscode → /workspaces/numerical-calculation/src (main) $ ./fibonacc
```

```
func(0.381966)
```

```
1: f(0.381966) = 0.00461938
```

```
func(0.618034)
```

```
2: f(0.618034) = 0.0924367
```

```
func(0.236068)
```

```
3: f(0.236068) = 0.0060734
```

```
func(0.472136)
```

```
4: f(0.472136) = 0.025007
```

```
func(0.326238)
```

```
5: f(0.326238) = 0.000149767
```

```
func(0.291796)
```

```
6: f(0.291796) = 0.000493015
```

```
func(0.347524)
```

```
7: f(0.347524) = 0.00112387
```

```
func(0.313082)
```

```
8: f(0.313082) = 8.42166e-07
```

```
func(0.304952)
```

```
9: f(0.304952) = 8.1872e-05
```

```
func(0.318107)
```

```
10: f(0.318107) = 1.68699e-05
```

```
func(0.309977)
```

```
11: f(0.309977) = 1.61871e-05
```

```
func(0.315002)
```

```
12: f(0.315002) = 1.00337e-06
```

```
func(0.311896)
```

```
13: f(0.311896) = 4.42655e-06
```

```
func(0.313815)
```

```
14: f(0.313815) = 3.4062e-08
```

```
func(0.314269)
```

```
15: f(0.314269) = 7.21165e-08
```

```
func(0.313535)
```

```
16: f(0.313535) = 2.15846e-07
```

```
func(0.313989)
```

```
17: f(0.313989) = 1.31975e-10
```

```
func(0.314095)
18: f(0.314095) = 9.11533e-09
func(0.313922)
19: f(0.313922) = 6.02124e-09
func(0.314029)
20: f(0.314029) = 8.62339e-10
func(0.313963)
21: f(0.313963) = 1.35005e-09
func(0.314004)
22: f(0.314004) = 1.68972e-11
func(0.314014)
23: f(0.314014) = 1.89529e-10
func(0.313998)
24: f(0.313998) = 3.35523e-12
func(0.313994)
25: f(0.313994) = 3.07548e-11
func(0.314)
26: f(0.314) = 1.57333e-13
func(0.314002)
27: f(0.314002) = 3.54282e-12
func(0.314)
28: f(0.314) = 1.19814e-13
func(0.313999)
29: f(0.313999) = 1.18578e-12
x = 0.314
```

最小値付近で 29 回の関数評価を行い、収束値として $x \doteq 0.314$ を得た。

2. 考察

黄金分割法は、探索区間を黄金比で分割することで効率的に探索を進める手法であり、収束の速さが特徴である。今回の実行結果でも、フィボナッチ探索法よりも少ない関数評価で同程度の精度を得ることができた。

フィボナッチ探索法は、探索区間をフィボナッチ数列に基づいて分割するため、事前に探索回数を決めておけるという特徴がある。一方で、今回の実験のように終了条件を同じにすると、黄金分割法より関数評価回数が増える傾向がある。

どちらの手法も収束結果自体はほとんど差がなく、精度の点では同等であるとい

える。ただし実装の容易さと汎用性を考えると、黄金分割法の方が実用的であると考えられる。

3. 結論

黄金分割法とフィボナッチ探索法はいずれも有効な一変数探索手法であり、最小値 $x \doteq \mathbf{0.314}$ を得ることができた。

ただし、今回の結果では黄金分割法の方が少ない関数評価で収束しており、効率性の面で優れていることが分かった。